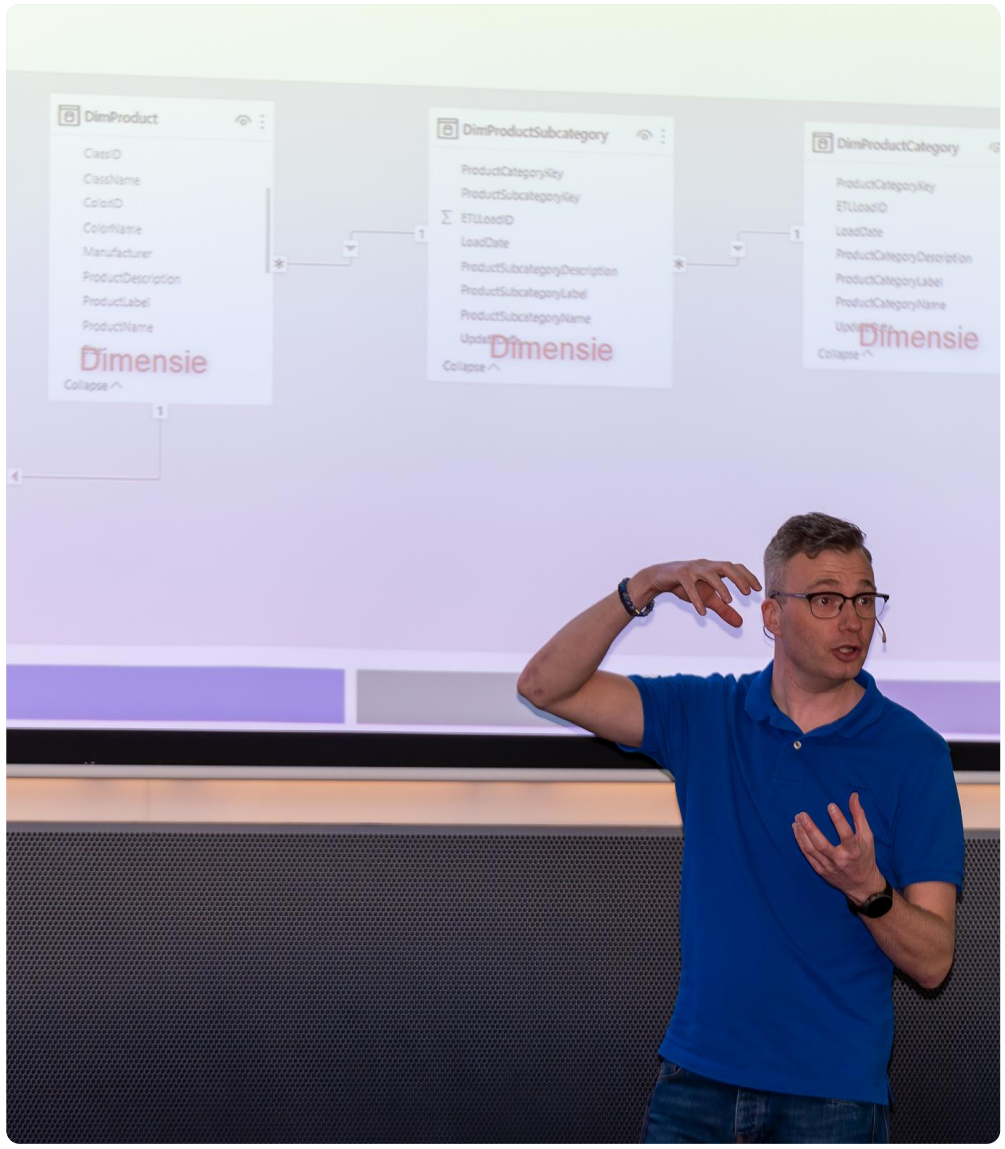
dutchBIgeek

Modeling Power BI with Code: Unlocking the Power of Semantic Link

Build, explore and clean up a model, live

14-10-2026



Thank you, partners

















































Peter van den Bos

Semantic Link

Data Modeling & DAX Specialist
Freelance Trainer & Speaker
Founder @ DutchBIGeek

- dutchbigeek.nl
- linkedin.com/in/petervdbos
- github.com/PetervandenBos
- sessionize.com/peter-van-den-bos
- youtube.com/@DutchBIgeek
- peter@dutchbigeek.nl



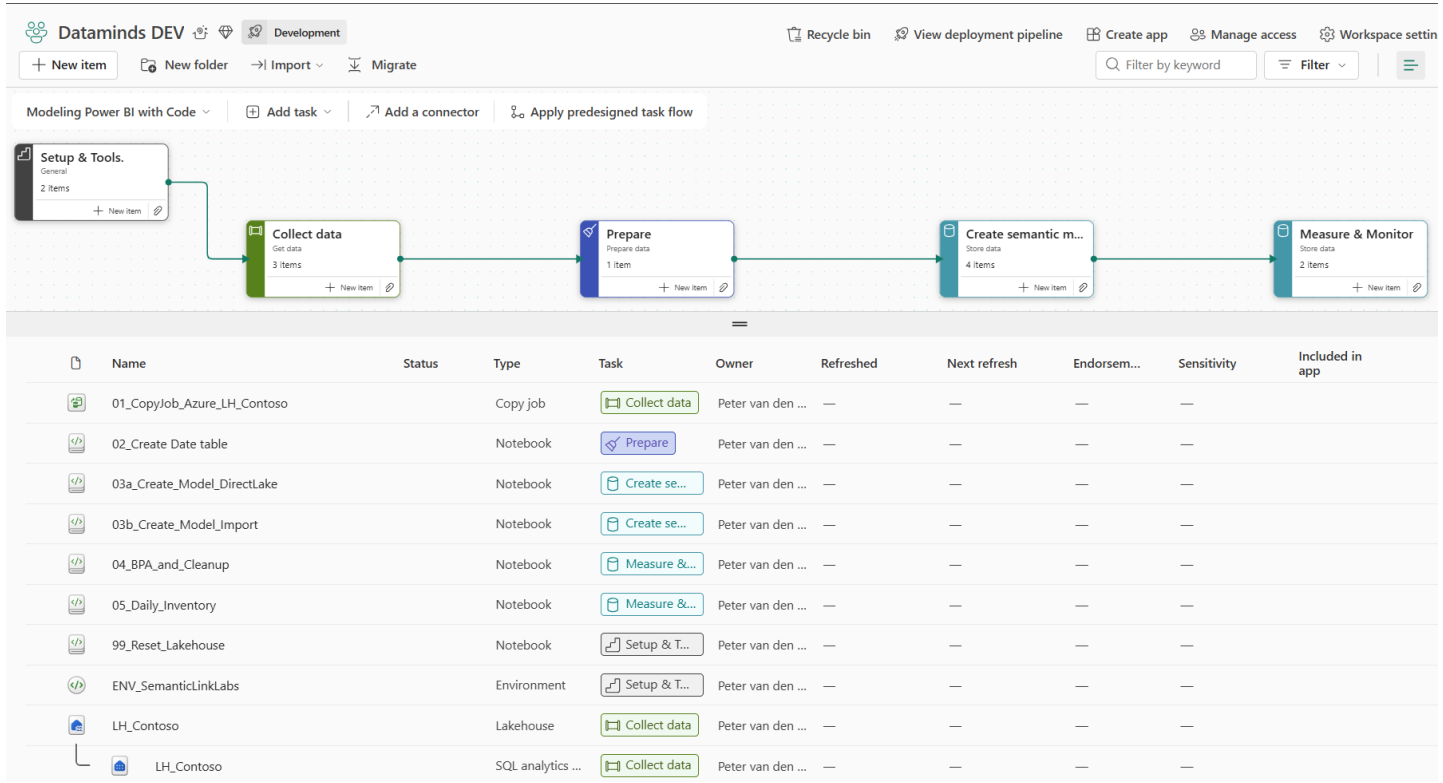
🧠 **Geek by design**

⚡ *Because performance isn't optional*

dutchBIgeek

One workspace, one flow

Semantic Link



Numbered in demo order: 01 → 05, with 99 as the reset

Setup & Tools

- Environment, reset notebook

Collect data

- Copy job: Azure SQL → LH_Contoso

Prepare

- DimDate from the fact data

Create semantic models

- Direct Lake and Import, same code

Measure & Monitor

- BPA, VertiPaq, cleanup, inventory

dutchBIgeek

Semantic Link

Semantic Link

sempy

Built into Fabric

Reads: models, tables, measures

Runs DAX against the model

semantic-link-labs

Open source, on top of sempy

Writes: tables, relationships, measures (TOM)

BPA and VertiPaq Analyzer

Why not Power Query? Power Query shapes the data inside one model. Semantic Link changes the model itself, in code: repeatable, in Git, for many models.

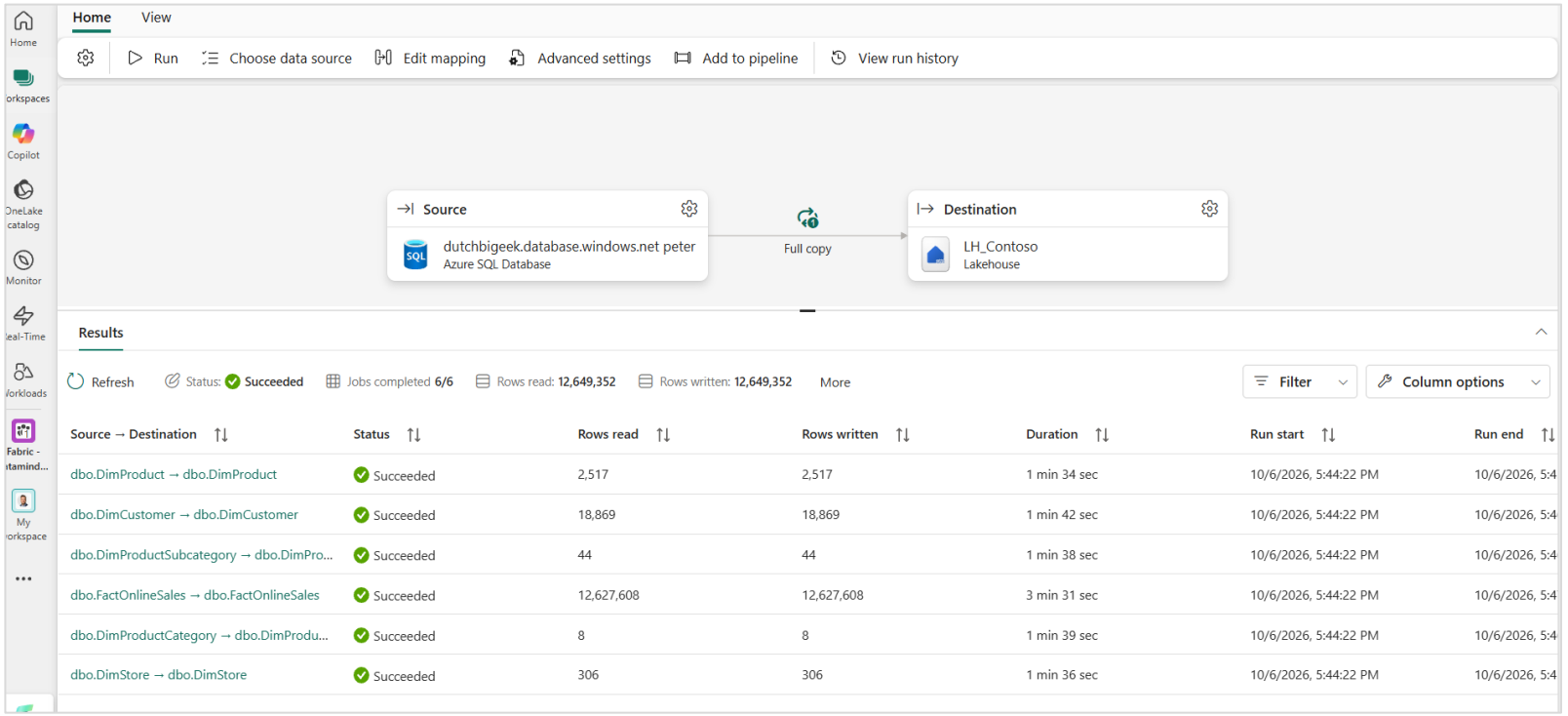
Collect data

Semantic Link

Step 1 · Copy job from Azure SQL into the lakehouse

Copy job: six tables, one full copy

Semantic Link



12.6M

rows in FactOnlineSales

3:31

first run on F2 (min)

2:37

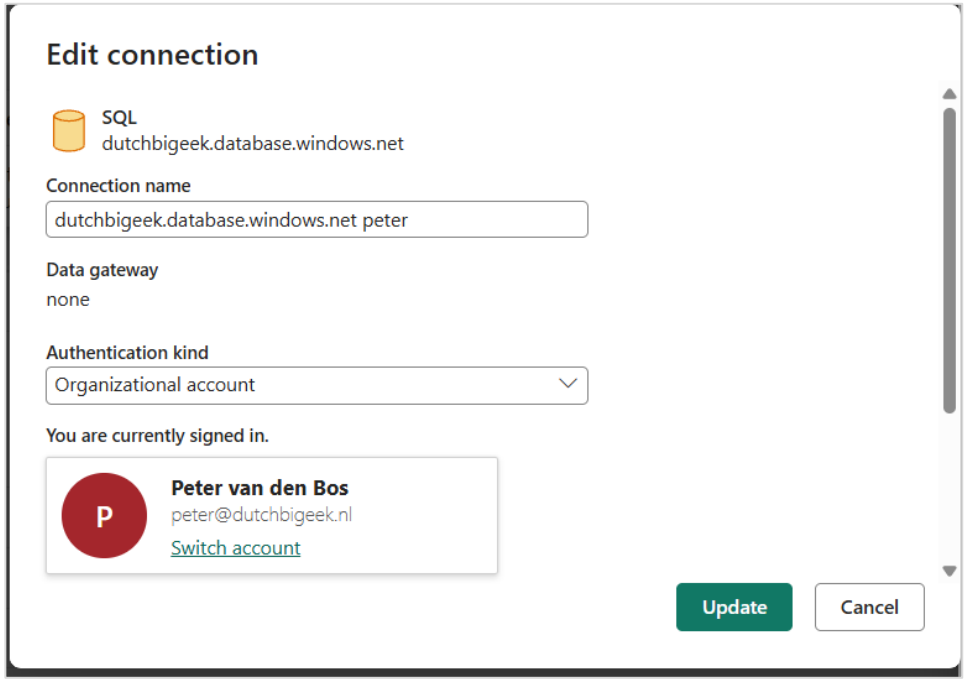
second run, capacity warm

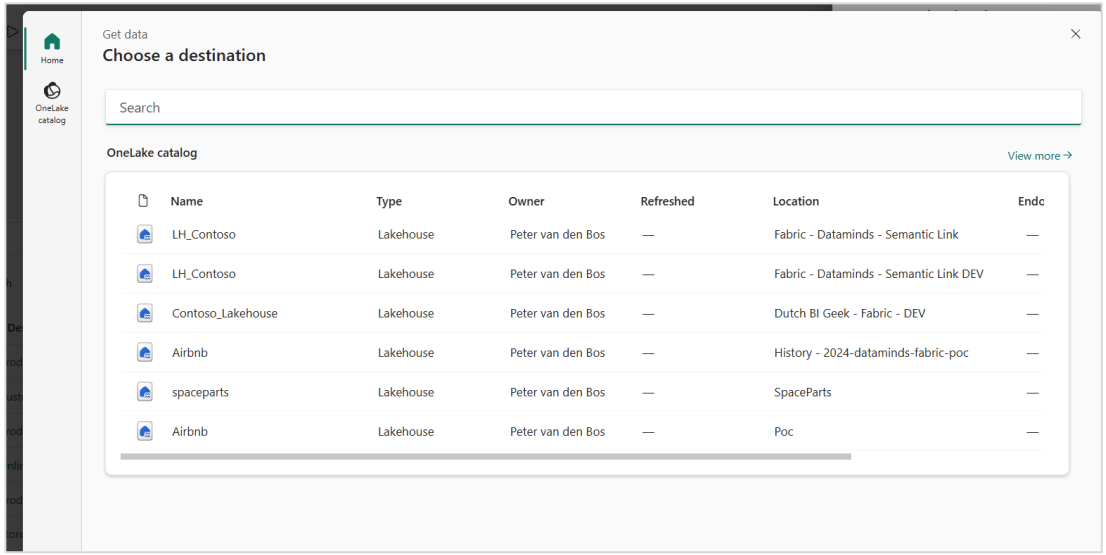
- Source: Azure SQL Database, no gateway needed
- Destination: LH_Contoso as Delta tables, write mode Overwrite

dutchBIgeek

Connection and destination

Semantic Link



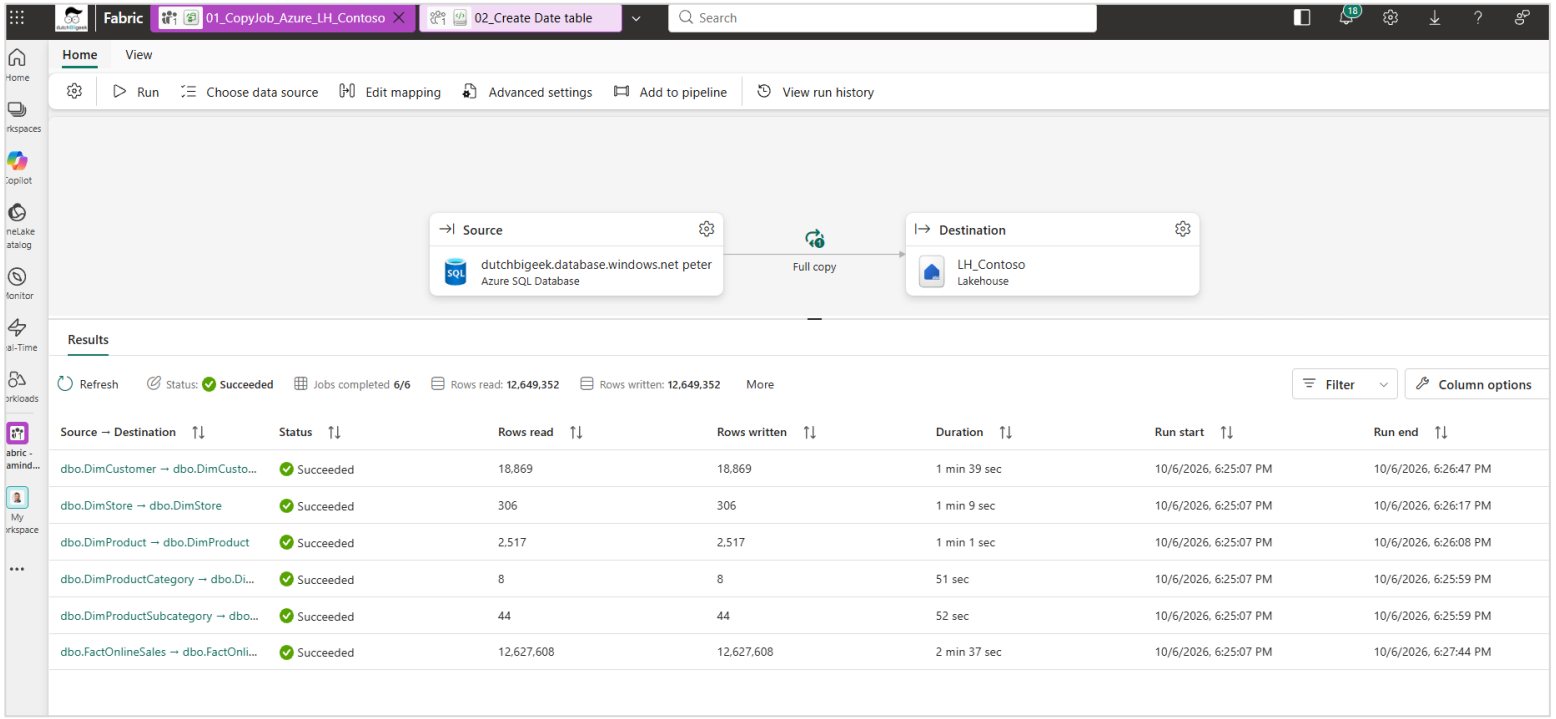


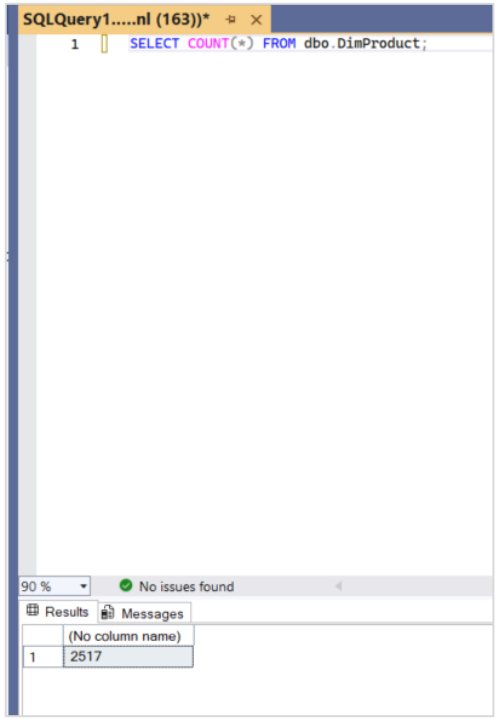
- No gateway: Azure SQL is reachable from the cloud
- Production: service principal or workspace identity
- Same lakehouse name in PROD and DEV: IDs tell the truth

dutchBIgeek

Rerun safely: Overwrite, not Append

Semantic Link



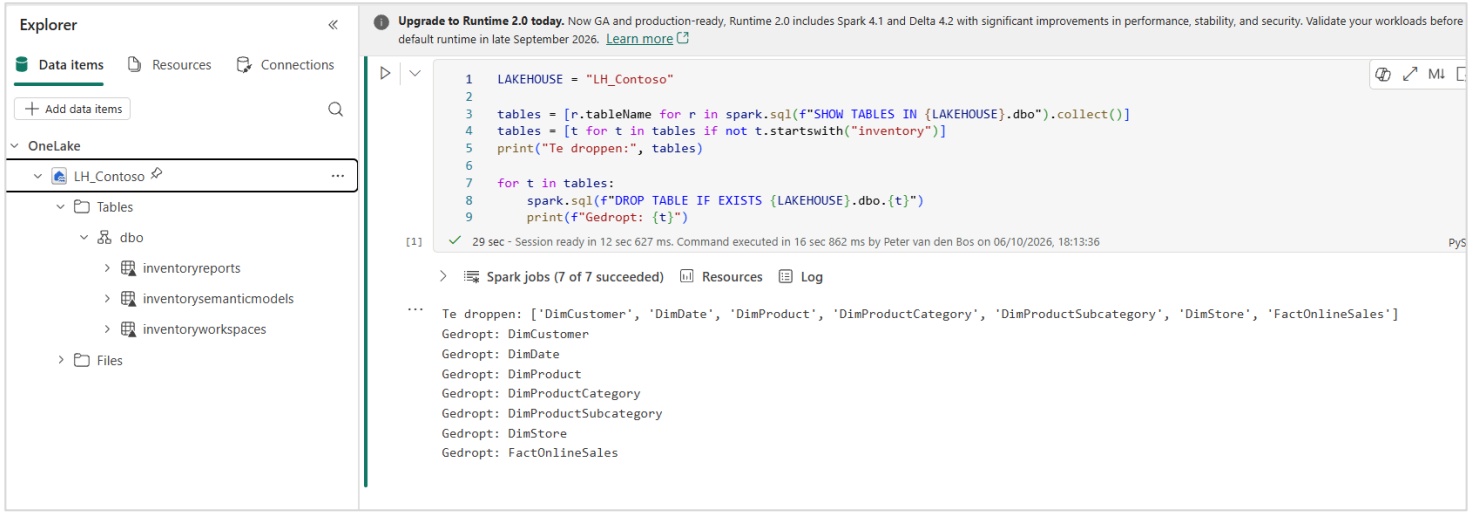


- Append on a full copy doubles every row: broken keys, double sales
- Check after a rerun: SELECT COUNT(*) on a dimension, 2,517 = no duplicates

dutchBIgeek

Reset before every test run

Semantic Link



99_Reset_Lakehouse

- Drops the Contoso tables, keeps the inventory history
- Keeps the lakehouse: a new one gets a new ID and breaks the copy job and notebooks
- SQL endpoint is read-only: drop from a notebook

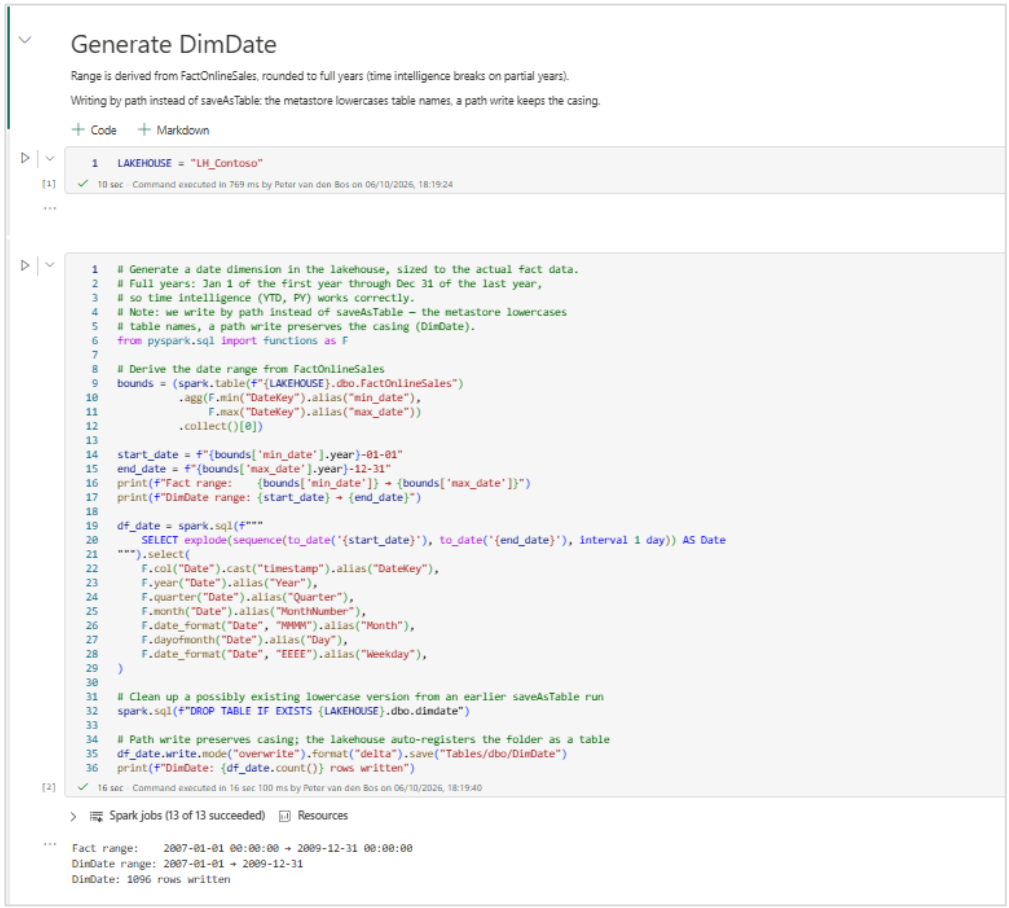
Prepare

Semantic Link

Step 2 · A date table generated from the data itself

DimDate, sized to the fact data

Semantic Link



- Range read from FactOnlineSales, full years
- 1,096 rows: 2007-01-01 → 2009-12-31
- DateKey as timestamp, the same type as the fact
- Path write keeps the casing: DimDate, not dimdate
- Runs after the copy job: the range depends on the fact

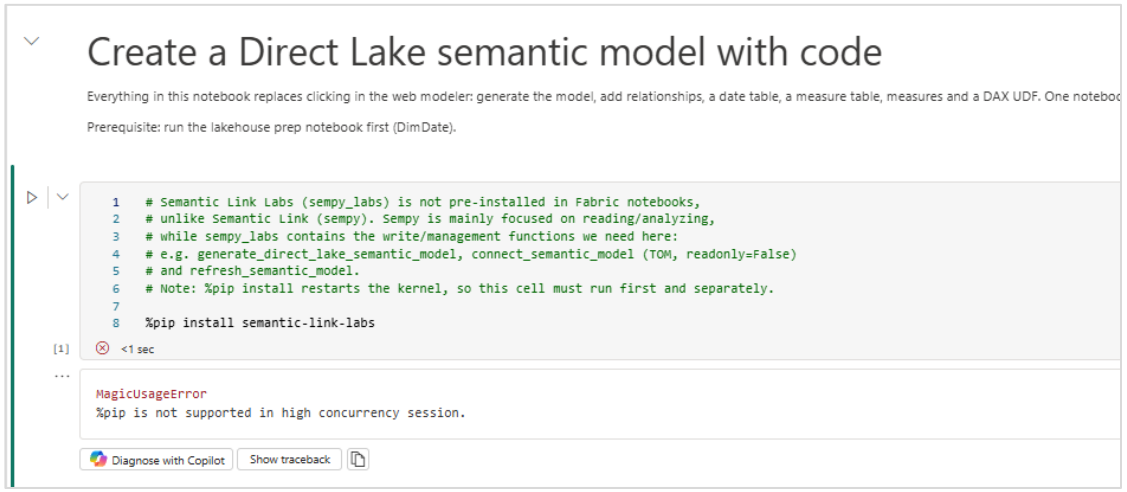
Setup & Tools

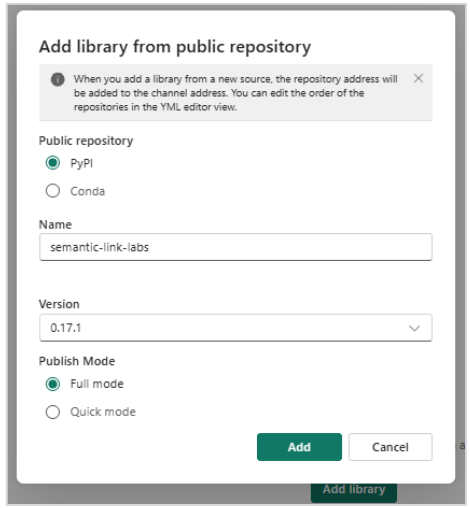
Semantic Link

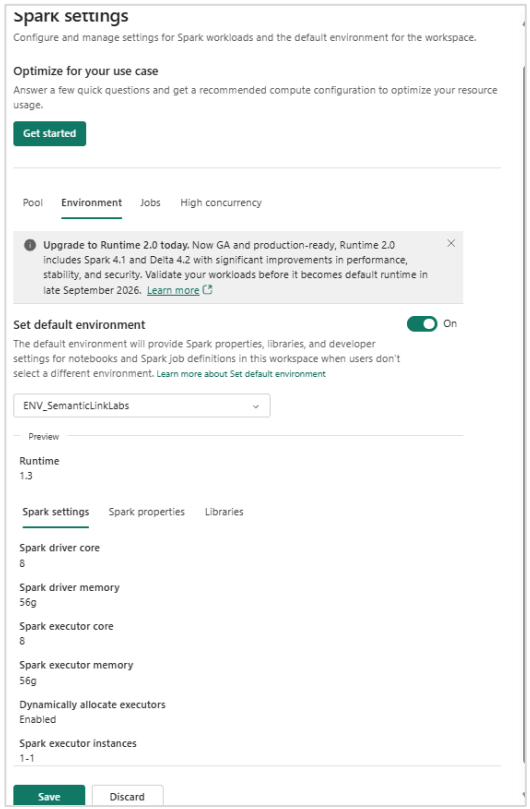
Step 3 · Environment, compute and capacity: the part nobody demos

%pip doesn't run in a shared session

Semantic Link



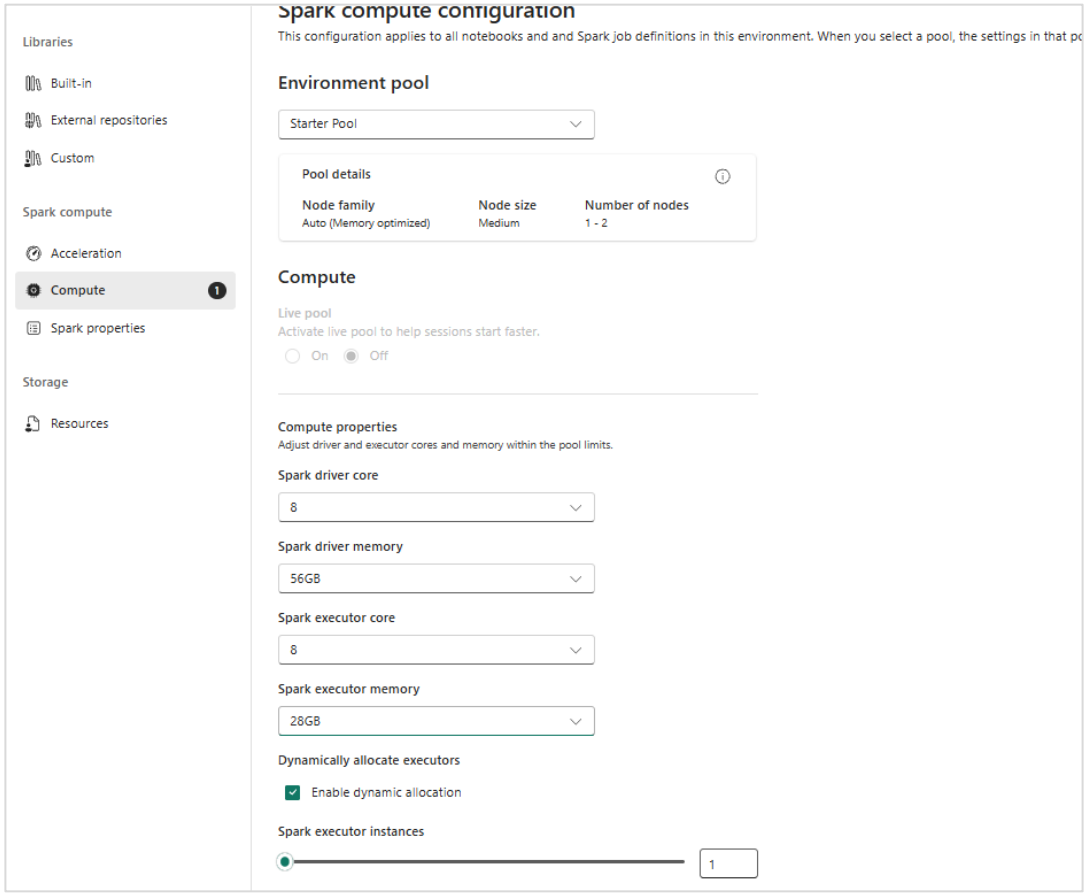




- High concurrency: up to 5 notebooks share one session, but refuse %pip
- ENV_SemanticLinkLabs: semantic-link-labs 0.17.1, Full mode, pinned
- Workspace default: no install cell, no kernel restart mid-run

Size the compute to the capacity

Semantic Link



16

vCores requested: 8 driver + 8 executor

4

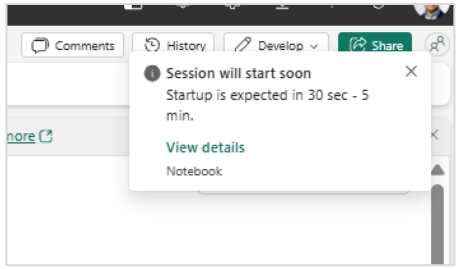
Spark vCores on an F2 (12 with burst)

- Driver and executor: 4 cores / 28 GB, dynamic allocation off
- Semantic Link mostly calls the model: small Spark is enough

F2 works, but it is slow

Semantic Link





430

TooManyRequestsForCapacity

30 s – 5 min

session start, custom environment

2 CU

Azure quota: F4 or F8 blocked

I tried to do everything properly on F2, but capacity is scarce. The Fabric trial gave me an F64: that solved it.

dutchBIgeek

Create semantic models

Semantic Link

Step 4 · Direct Lake and Import, built with the same code

What happens when you write

Semantic Link

1. Connect

Open the model with readonly=False. TOM over XMLA.

2. Change

Add tables, relationships, measures. Nothing is sent yet.

3. Save

End of the with-block: all changes saved at once

4. Recalculate

New relationship: CalculationNeeded. Refresh first, then DAX can use it.

And refresh_tom_cache, or sempy still shows the old model.

Framing is not a warm model

Semantic Link

Direct Lake refresh = framing

Points the model at the current Delta files

No data loaded yet

First query loads the columns: slow

Import refresh = processing

Reads all the data

Compresses it into memory

First query is fast

Fair comparison: refresh plus the first query.

Same code, same numbers

Semantic Link

```
# Test with a DAX query: the numbers must match the Direct Lake model exactly.

fabric.evaluate_dax(DATASET, workspace=WORKSPACE, dax_string="""
EVALUATE
SUMMARIZECOLUMNS(
    DimDate[Year],
    "Total Sales", [Total Sales],
    "Margin classic", [Margin %],
    "Margin via UDF", [Margin % (UDF)]
)
""")
```

[11] ✓ 1 sec - Command executed in 1 sec 579 ms by Peter van den Bos on 06/10/2026, 19:31:22

	DimDate[Year]	[Total Sales]	[Margin classic]	[Margin via UDF]
0	2007	1108917269.99	0.577027	0.577027
1	2008	934403814.178	0.568747	0.568747
2	2009	940877836.936	0.569861	0.569861

	Direct Lake	Import
Refresh	6 s (framing)	1:30 (processing)
Total Sales 2007	1,108,917,269.99	1,108,917,269.99
Margin 07 · 08 · 09	57.7 · 56.9 · 57.0%	identical

- Relationships, measures and UDF: the same cells in both notebooks
- Import fills all the size statistics: that's why the cleanup runs there

Measure & Monitor

Semantic Link

Step 5 · Measure, clean up, measure again, then keep watching

Find what nobody uses

Semantic Link

```python
# The ten most expensive columns, with their share of the table and of the model.
# This is where cleanup pays off.

df_size = fabric.list_columns(DATASET, workspace=WORKSPACE, extended=True)

model_total = df_size["Total Size"].sum()
table_totals = df_size.groupby("Table Name")["Total Size"].transform("sum")
df_size["% of Table"] = (df_size["Total Size"] / table_totals * 100).round(2)
df_size["% of Model"] = (df_size["Total Size"] / model_total * 100).round(2)

before_size = model_total
print(f"Model size before cleanup: {before_size / 1024**2:,.0f} MB")

df_size[["Table Name", "Column Name", "Total Size", "Data Size", "Dictionary Size",
         "Column Cardinality", "% of Table", "% of Model"]] \
    .sort_values("Total Size", ascending=False).head(10)
```

[4] ✓ 1 sec - Command executed in 1 sec 648 ms by Peter van den Bos on 06/10/2026, 19:38:18

```
Model size before cleanup: 216 MB
```

	Table Name	Column Name	Total Size	Data Size	Dictionary Size	Column Cardinality	% of Table	% of Model
34	FactOnlineSales	OnlineSalesKey	101021168	50510568	144	12627608	45.29	44.57
39	FactOnlineSales	SalesOrderNumber	85168900	15561184	56213108	1674320	38.19	37.57
38	FactOnlineSales	CustomerKey	11366668	10615640	600036	18869	5.1	5.01
37	FactOnlineSales	ProductKey	7943344	7847296	75872	2516	3.56	3.5
35	FactOnlineSales	DateKey	7108028	7056448	42764	1096	3.19	3.14
41	FactOnlineSales	UnitCost	5214072	5199656	10528	480	2.34	2.3
42	FactOnlineSales	UnitPrice	5205816	5192120	10240	426	2.33	2.3
8	DimCustomer	EmailAddress	910229	37104	725205	18484	30.15	0.4
1	DimCustomer	CustomerKey	788900	37872	600036	18869	26.13	0.35
9	DimCustomer	AddressLine1	654381	37104	514813	12802	21.67	0.29

Find what nobody uses

A column is in use when it's in a relationship, referenced by a measure, or used as sort-by column. Everything else is a candidate for removal.

```python
# Collect every column the model actually uses, then list the rest as candidates.
# The big ones at the top are the expensive dead weight.

used = set()

rels = fabric.list_relationships(DATASET, workspace=WORKSPACE)
for _, r in rels.iterrows():
    used.add((r["From Table"], r["From Column"]))
    used.add((r["To Table"], r["To Column"]))

with connect_semantic_model(dataset=DATASET, workspace=WORKSPACE, readonly=True) as tom:
    expressions = " ".join(m.Expression for t in tom.model.Tables for m in t.Measures)
    for t in tom.model.Tables:
        for c in t.Columns:
            if re.search(rf"\[{re.escape(c.Name)}\]", expressions):
                used.add((t.Name, c.Name))
            if c.SortByColumn is not None:
                used.add((t.Name, c.SortByColumn.Name))

df_cand = df_size[
    ~df_size.apply(lambda r: (r["Table Name"], r["Column Name"]) in used, axis=1)
    & ~df_size["Column Name"].str.startswith("RowNumber")
]
df_cand[["Table Name", "Column Name", "Total Size", "Column Cardinality"]] \
    .sort_values("Total Size", ascending=False)
```

[5] ✓ 2 sec - Command executed in 2 sec 378 ms by Peter van den Bos on 06/10/2026, 19:38:25

	Table Name	Column Name	Total Size	Column Cardinality
34	FactOnlineSales	OnlineSalesKey	101021168	12627608
39	FactOnlineSales	SalesOrderNumber	85168900	1674320
8	DimCustomer	EmailAddress	910229	18484
9	DimCustomer	AddressLine1	654381	12802
10	DimCustomer	Phone	358871	8890
15	DimProduct	ProductName	149067	2517
16	DimProduct	ProductDescription	84385	809

- Used = relationship, measure expression or sort-by column, read from TOM
- OnlineSalesKey + SalesOrderNumber: 82% of the model, serving nothing
- A candidate list, not a kill list: report usage isn't in the model

Why two columns cost 82%

Semantic Link

Distinct values, not rows

VertiPaq stores every distinct value once. Rows point to it.

Almost unique = no compression

An ID per row gives nothing to compress. Text IDs also make a big dictionary.

Check it

VertiPaq Analyzer: cardinality, dictionary size, data size.

The most expensive columns were the ones nobody used.

BPA again: a fix can trigger another rule

Semantic Link

```
# BPA again: the hidden keys and summarization warnings should be gone.

labs.run_model_bpa(dataset=DATASET, workspace=WORKSPACE)
```

[9] ✓ 6 sec - Command executed in 6 sec 267 ms by Peter van den Bos on 06/10/2026, 19:40:36

Formatting ⚠ (9) + ℹ (5) | **Maintenance** ℹ (38) + ⚠ (1) | **Performance** ⚠ (16)

Rule Name	Object Type	Object Name	Severity
Mark primary keys	Column	'DimCustomer'[CustomerKey]	ℹ
Mark primary keys	Column	'DimProduct'[ProductKey]	ℹ
Mark primary keys	Column	'DimProductCategory'[ProductCategoryKey]	ℹ
Mark primary keys	Column	'DimProductSubcategory'[ProductSubcategoryKey]	ℹ
Mark primary keys	Column	'DimStore'[StoreKey]	ℹ
Percentages should be formatted with thousands separators and 1 decimal	Measure	Margin %	⚠
Percentages should be formatted with thousands separators and 1 decimal	Measure	Margin % (UDF)	⚠
Provide format string for 'Date' columns	Column	'DimCustomer'[BirthDate]	⚠
Provide format string for 'Date' columns	Column	'DimCustomer'[DateFirstPurchase]	⚠
Provide format string for 'Date' columns	Column	'DimDate'[DateKey]	⚠
Provide format string for 'Date' columns	Column	'FactOnlineSales'[DateKey]	⚠
Relationship columns should be of integer data type	Relationship	'FactOnlineSales'[DateKey] -> 'DimDate'[DateKey]	⚠
Whole numbers should be formatted with thousands separators and no decimals	Measure	Total Cost	⚠
Whole numbers should be formatted with thousands separators and no decimals	Measure	Total Sales	⚠

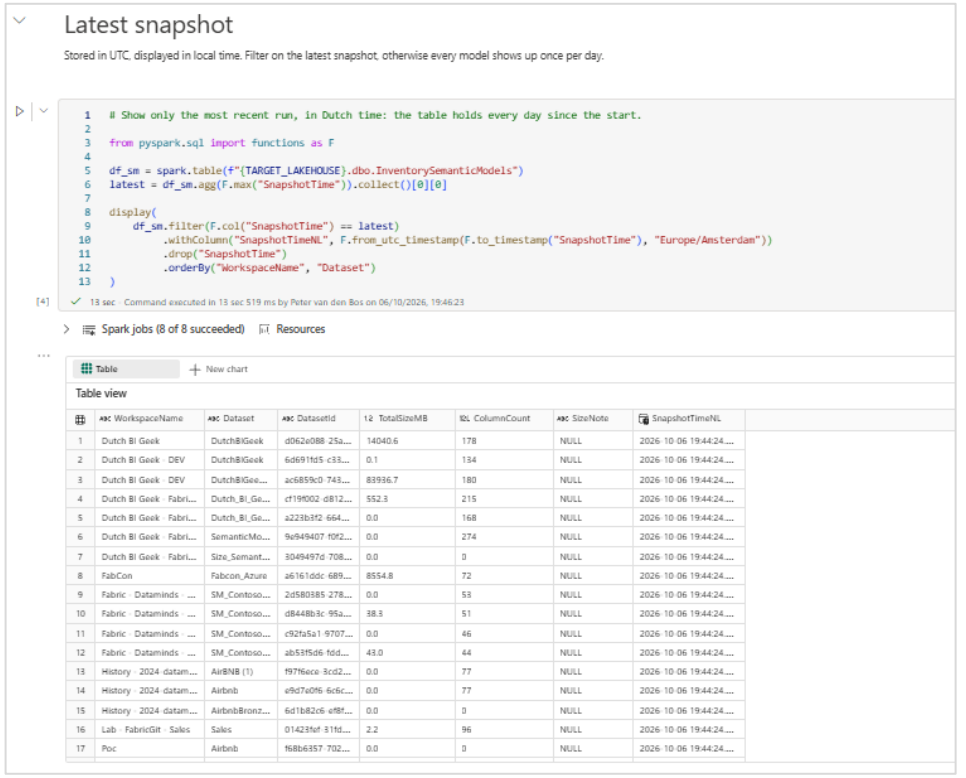
Category	Before	After
Formatting	27 ⚠ + 17 ℹ	9 ⚠ + 5 ℹ
Maintenance	52 ℹ + 1 ⚠	38 ℹ + 1 ⚠
Performance	4 ⚠	**16 ⚠**

- Hiding 12 keys triggers 'IsAvailableInMdx false on hidden columns'
- That's why you measure again: BPA is advice, not law

Daily inventory as monitoring

Semantic Link





- Workspaces, models, sizes and reports appended to Delta, one snapshot per run
- Locked system workspaces are logged and skipped, the run continues

dutchBIgeek

Review cells: demo yes, daily run no

Semantic Link

```
REVIEW = False   # True when presenting

# ... collect and write ...

if REVIEW:
    display(latest_snapshot)
    labs.vertipaq_analyzer(DATASET, WORKSPACE)
```

13 s

'Latest snapshot' display

19 s

VertiPaq Analyzer

9 s

BPA output

- In every demo I show a review: a table, a chart, a check
- In a notebook that runs every day, nobody looks at it
- It still costs process time and capacity, every run
- Keep it behind a flag: on for the demo, off for the schedule

Takeaways

Semantic Link

Build it like production, show it like a demo.

1. Number the items and draw the task flow: the workspace tells the story
2. Libraries in an environment, not %pip: high concurrency, no surprises
3. Every write cell idempotent: a double click can't break the model
4. Same code, two storage modes: compare refresh plus first query
5. Measure, clean up, measure again: 216 → 38 MB, same answers
6. Size compute to the capacity; F2 works, the trial F64 saves time

How did I do?

Semantic Link

Scan to rate this session.



dutchBIgeek

If you have questions or insights, please contact me !



Peter van den Bos

Business Intelligence Consultant

peter@dutchbigeek.nl

+31 6 13760795

https://www.linkedin.com/in/petervdbos/

dutchbigeek.nl